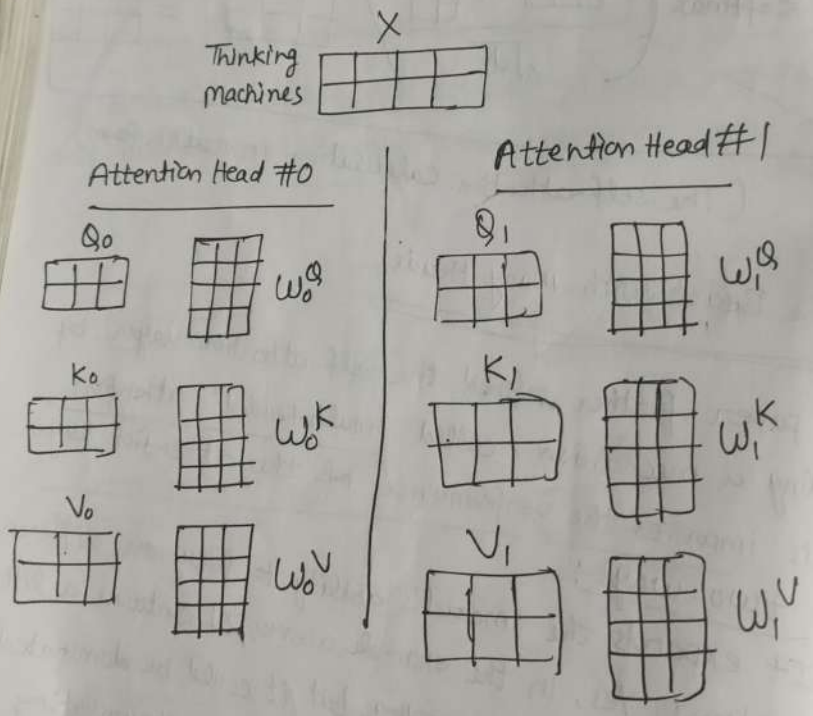
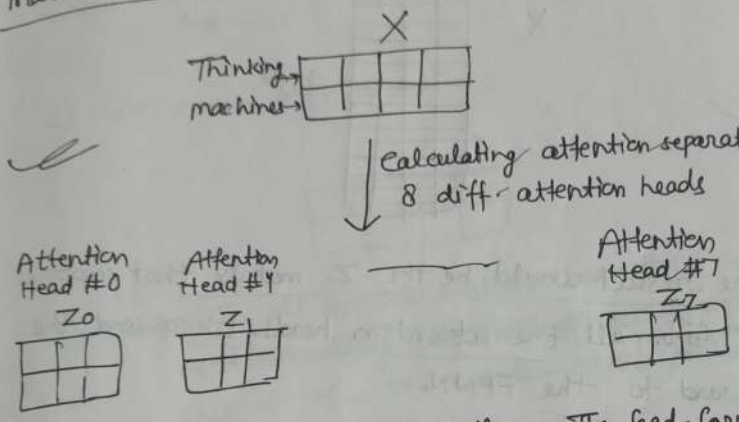
② It gives the attention layer multiple "representation subspaces". As we'll see next, with multi-headed attention we have not only one, but multiple sets of Query/Key/Value weight matrices (The Transformer uses 8 attention heads, so we end up with 8 sets ~~of~~ for each encoder/decoder). Each of these sets is randomly initialized. Then, after training, each set is used to project the input embedding (or vectors from lower encoders/decoders) into a different representation subspace.



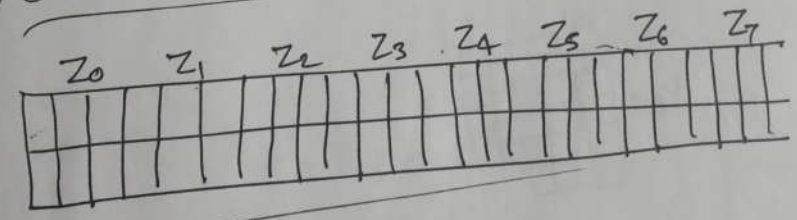
If we do the same self-attention calculation, we outlined above, just 8 diff. times with diff weight matrices, we end up with 8 diff. Z matrices.



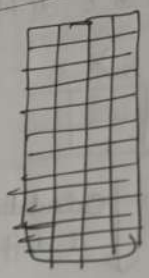
This leaves us with a bit of a challenge. The feed-forward layer is not expecting 8 matrices. - it's expecting a single matrix (a vector for each word). So, we need a way to condense these 8 down into a single matrix

How do we do that? We concat the matrices then multiply them by an additional weights matrix W^O.

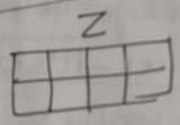
1) Concatenate all the attention heads:



2) multiply with a weight matrix W^O that was trained jointly with the model.

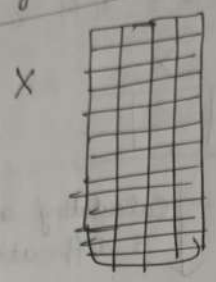


3) The result would be the Z matrix that captures info^m from all the attention heads. We can send this forward to the FFNN.

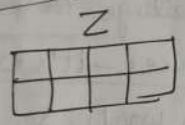


↳ That's pretty much all there is to multi-headed self-attent. It's quite a handful of matrices, I realize. Let me try to put them all in one visual so we can look at them in one place.

2) multiply with a weight matrix W^O that was trained jointly with the model.

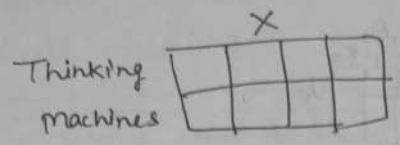


3) The result would be the Z matrix that captures info^m from all the attention heads. We can send this forward to the FFNN.

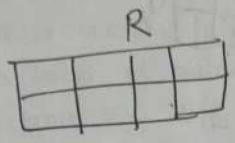


↳ That's pretty much all there is to multi-headed self-attent.
It's quite a handful of matrices, I realize. Let me try to put them all in one visual so we can look at them in one place.

1) This is our input sentence*

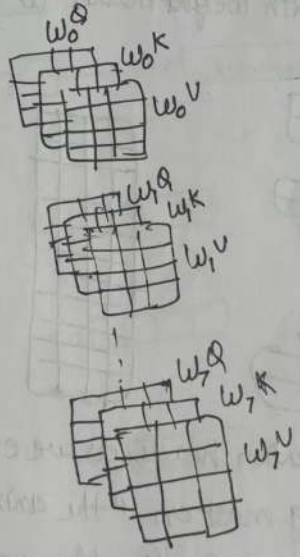


2) We embed each word*

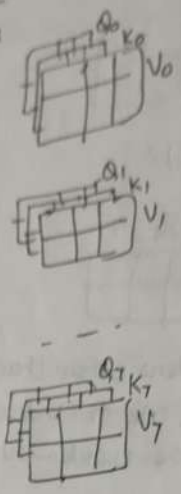


(*) ⇒ In all encoders other than #0, we don't need embedding. ~~We start directly with the output of the encoder right below this~~

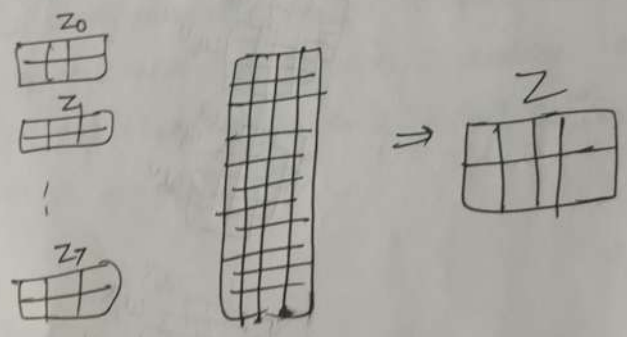
3) Split into 8 heads. We multiply X or R with weight matrices



4) Calculate attention using the resulting Q/K/V matrices



5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



→ With multi-attention heads, as we encode the word "it", one attention head is focusing most on "the animal", while another is focusing on "tired" — In a sense, the model's representation of the word "it" bakes in some of the representation of both "animal" & "tired".

⇒ Repre
Posi

One thing
so far is a
sequence.
each inpu
the model
each w
The intu
embeddi
vectors
during

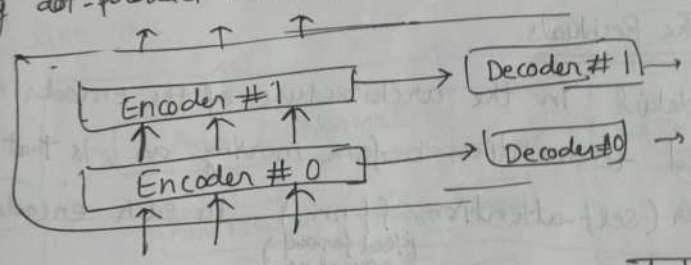
Embe
with

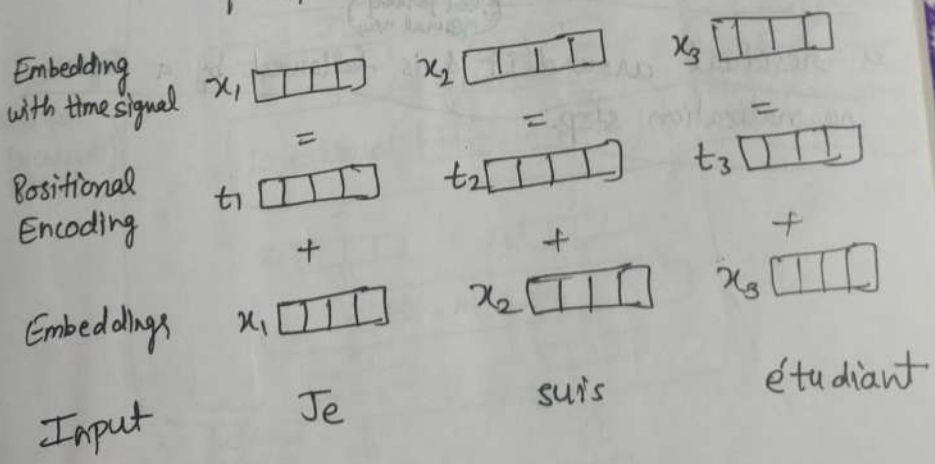
Posi
En

En

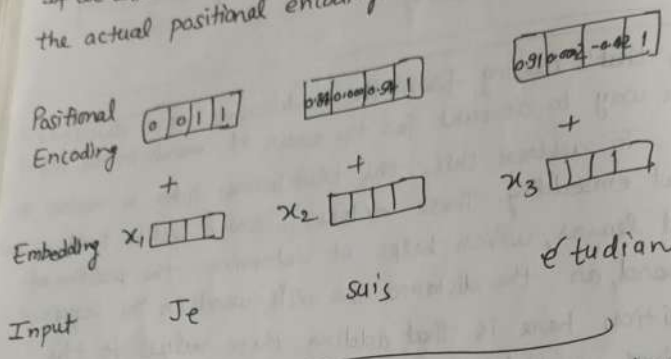
⇒ Representing the Order of the sequence using Positional Encoding.

One thing that's missing from the model as we have described it so far is a way to account for the order of words in the input sequence. To address this, the transformer adds a vector to each input embedding. These vectors follow a specific pattern that the model learns, which helps it determine the position of each word, or the distance b/w diff. words in the sequence. The intuition here is that adding these values to the embeddings provides meaningful distances b/w the embedding vectors once they're projected into Q/K/V vectors & during dot-product attention.





If we assumed the embedding has a dimensionality of 4, the actual positional encodings would look like this:

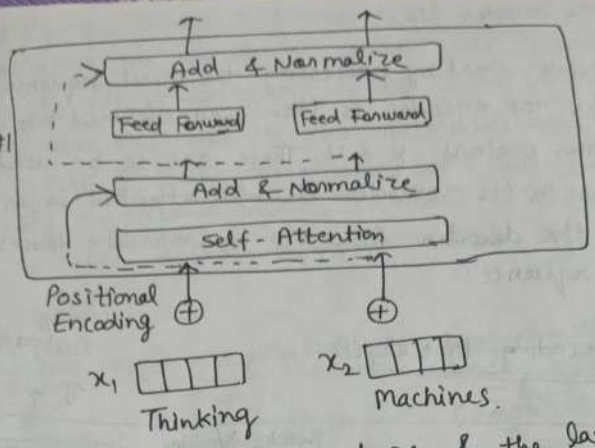


A real example of positional encoding with a toy embedding size of 4.

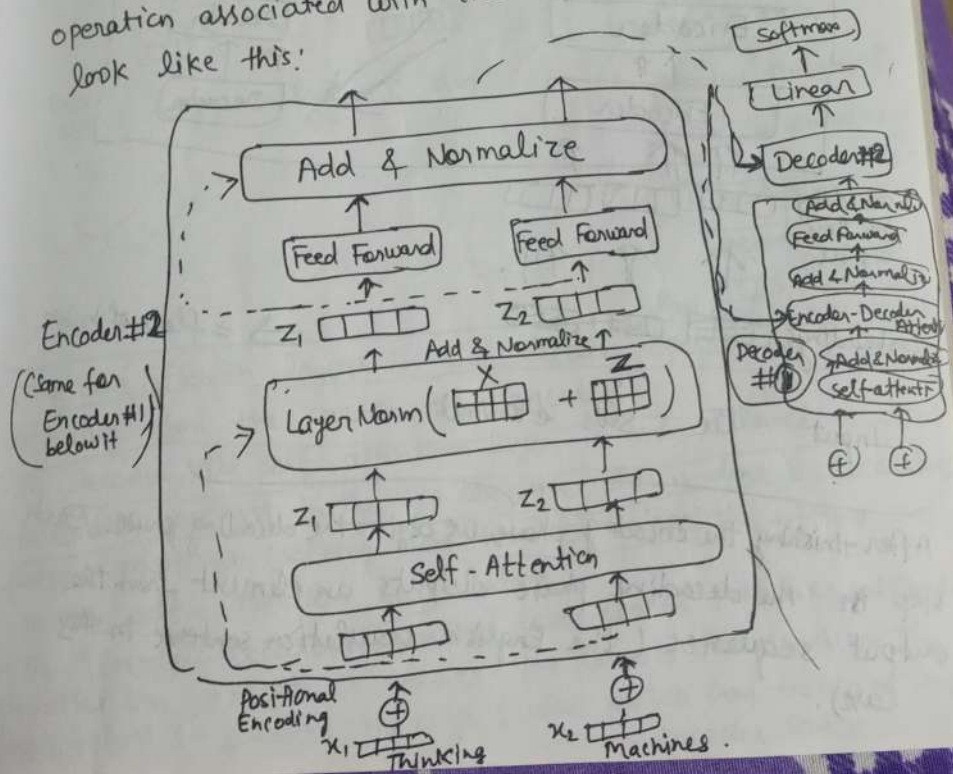
The Residuals.

One detail in the architecture of the encoder that we need to mention before moving on, is that each sub-layer (self-attention, ffnn) in each encoder has a residual around it & is followed by a layer-normalization step.

(feed forward neural n/w)



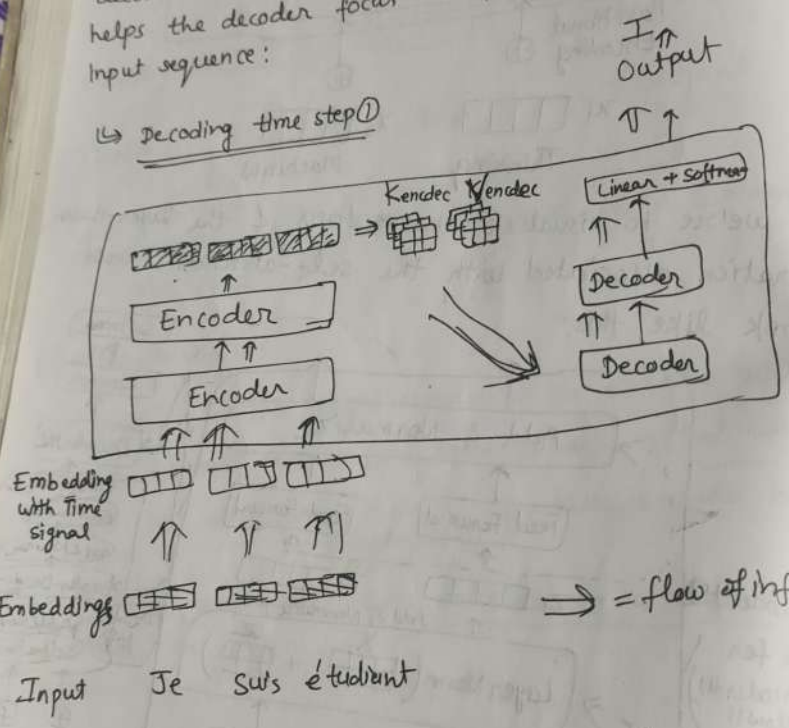
If we're to visualize the vectors & the layer-norm operation associated with the self-attention, it would look like this:



⇒ The Decoder Side

The encoder start by processing the input sequence. The o/p of the top encoder is then transformed into a set of attention vectors K & V. These are to be used by each decoder in its "encoder-decoder attention" layer which helps the decoder focus on appropriate places in the input sequence:

↳ Decoding time step①

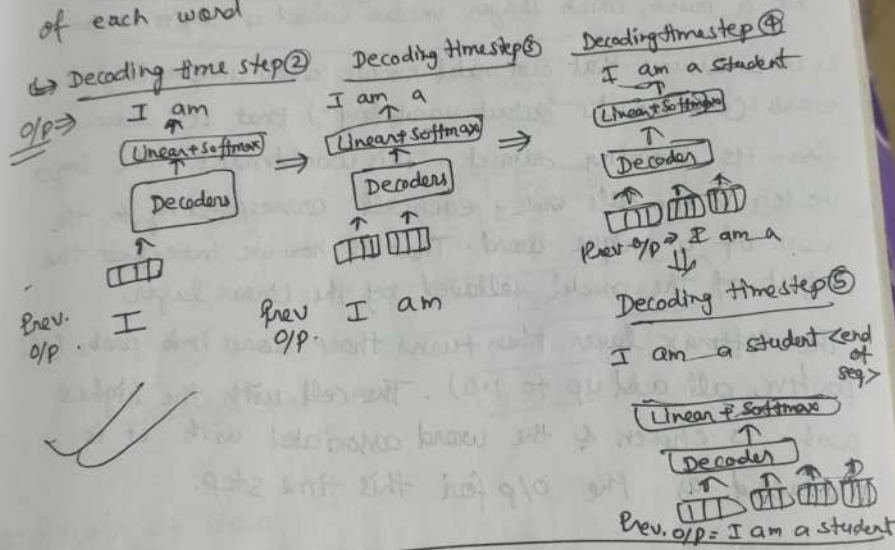


⇒ = flow of info^n

After finishing the encoding phase, we begin the decoding phase. Each step in the decoding phase outputs an element from the output sequence (the English translation sentence in this case).

The o/p
set of
g each
Rich
the

The following steps repeat the process until a special symbol is reached indicating the transformer decoder has completed its output. The o/p of each step is fed to the bottom decoder in the next time step, and the decoders bubble up their decoding results just like the encoders did. And just like we did with the encoder inputs, we embed & add positional encoding to those decoder inputs to indicate the position of each word



↳ The self attention layers in the decoder operate in a slightly diff. way than the one in the encoder:
In decoder, the self-attention layer is only allowed to attend to earlier positions in the o/p sequence. This is done by masking future positions. (setting them to -inf) before the softmax step in the self-attention calculation.

↳ The "Encoder-Decoder Attention" layer works just like multiheaded self-attention, except it creates its Queries matrix from the layer below it & takes the keys & values matrix from the o/p of the encoder stack.